

Exercise 1: Implementing the Singleton Pattern

//public static method to get the instance of the Logger class

```
public class Main {  
    public static void main(String[] args) {  
  
        Logger logger1 = Logger.getInstance();  
        logger1.log("Application Started");  
        Logger logger2 = Logger.getInstance();  
        logger2.log("User Logged In");  
        System.out.println("logger1 == logger2 : " + (logger1 == logger2));  
    }  
}
```

// Create a class named Logger that has a private static instance of itself.

```
class Logger {  
    private static Logger instance;  
    // constructor of Logger is private  
    private Logger() {  
        System.out.println("Logger instance created.");  
    }  
    //public static method to get the instance of the Logger class  
    public static Logger getInstance() {  
        //Code Implementation  
        if (instance == null) {  
            instance = new Logger();  
        }  
        return instance;  
    }  
    public void log(String message) {  
        System.out.println("[LOG]: " + message);  
    }  
}
```

Output

```
Logger instance created.  
[LOG]: Application Started  
[LOG]: User Logged In  
logger1 == logger2 : true
```

Exercise 2: Implementing the Factory Method Pattern

```
public class Main {
    public static void main(String[] args) {

        //Test the Factory Method Implementation
        DocumentFactory wordFactory = new WordFactory();
        Document wordDoc = wordFactory.createDocument();
        wordDoc.open();

        DocumentFactory pdfFactory = new PdfFactory();
        Document pdfDoc = pdfFactory.createDocument();
        pdfDoc.open();

        DocumentFactory excelFactory = new ExcelFactory();
        Document excelDoc = excelFactory.createDocument();
        excelDoc.open();
    }
}

// Create interfaces for different document types
interface Document {
    void open();
}

//Implement concrete classes for each document type
class WordDocument implements Document {
    public void open() {
        System.out.println("Opening Word Document");
    }
}
class PdfDocument implements Document {
    public void open() {
        System.out.println("Opening PDF Document");
    }
}
class ExcelDocument implements Document {
    public void open() {
        System.out.println("Opening Excel Document");
    }
}

//Create abstract factory class
abstract class DocumentFactory {
    public abstract Document createDocument();
}
```

```
//Create Concrete Factory Classes
class WordFactory extends DocumentFactory {
    public Document createDocument() {
        return new WordDocument();
    }
}
class PdfFactory extends DocumentFactory {
    public Document createDocument() {
        return new PdfDocument();
    }
}
class ExcelFactory extends DocumentFactory {
    public Document createDocument() {
        return new ExcelDocument();
    }
}
```

Output

```
Opening Word Document
Opening PDF Document
Opening Excel Document
```

Exercise 3: Implementing the Builder Pattern

```
public class Main {
    public static void main(String[] args) {
        // Testing the Builder Implementation
        Computer gamingPC = new Computer.Builder()
            .setCPU("Intel i9")
            .setRAM(32)
            .setStorage(1024)
            .build();
        Computer officePC = new Computer.Builder()
            .setCPU("Intel i5")
            .setRAM(16)
            .setStorage(512)
            .build();
    }
}
```

```

        System.out.println("Gaming PC:");
        gamingPC.display();
        System.out.println();
        System.out.println("Office PC:");
        officePC.display();
    }
}

```

```

class Computer {
    // Create a class Computer with attributes like CPU, RAM, Storage
    private String cpu;
    private int ram;
    private int storage;

    // Private Constructor
    private Computer(Builder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
        this.storage = builder.storage;
    }
    public void display() {
        System.out.println("CPU : " + cpu);
        System.out.println("RAM : " + ram + " GB");
        System.out.println("Storage : " + storage + " GB");
    }

    // Implement Builder Class
    public static class Builder {

        private String cpu;
        private int ram;
        private int storage;

        public Builder setCPU(String cpu) {
            this.cpu = cpu;
            return this;
        }
        public Builder setRAM(int ram) {
            this.ram = ram;
            return this;
        }
        public Builder setStorage(int storage) {
            this.storage = storage;
            return this;
        }
        public Computer build() {
            return new Computer(this);
        }
    }
}

```

```
}  
}
```

Output

Gaming PC:

CPU : Intel i9

RAM : 32 GB

Storage : 1024 GB

Office PC:

CPU : Intel i5

RAM : 16 GB

Storage : 512 GB

Exercise 4: Implementing the Adapter Pattern

//Implementing the Adapter Pattern

```
public class Main {  
    public static void main(String[] args) {  
  
        //Test the Adapter Implementation  
        PaymentProcessor paypal =  
            new PayPalAdapter(new PayPalGateway());  
        PaymentProcessor stripe =  
            new StripeAdapter(new StripeGateway());  
        paypal.processPayment(5000);  
        stripe.processPayment(10000);  
    }  
}
```

// Define Target Interface

```
interface PaymentProcessor {  
    void processPayment(double amount);  
}
```

//Implementing Adaptee Classes

```
class PayPalGateway {  
    public void makePayment(double amount) {  
        System.out.println("Payment " + amount +  
            " Rupees processed through PayPal");  
    }  
}
```

```

}
class StripeGateway {
    public void payAmount(double amount) {
        System.out.println("Payment " + amount +
            " Rupees processed through Stripe");
    }
}

```

```

//Implement Adapter Classes
class PayPalAdapter implements PaymentProcessor {
    private PayPalGateway paypal;
    public PayPalAdapter(PayPalGateway paypal) {
        this.paypal = paypal;
    }
    @Override
    public void processPayment(double amount) {
        paypal.makePayment(amount);
    }
}

```

```

class StripeAdapter implements PaymentProcessor {
    private StripeGateway stripe;
    public StripeAdapter(StripeGateway stripe) {
        this.stripe = stripe;
    }
    @Override
    public void processPayment(double amount) {
        stripe.payAmount(amount);
    }
}

```

Output

```

Payment 5000.0 Rupees processed through PayPal
Payment 10000.0 Rupees processed through Stripe

```

Exercise 5: Implementing the Decorator Pattern

```

//Testing the Decorator Implementation
public class Main{
    public static void main(String[] args){
        Notifier notifier = new EmailNotifier();
        System.out.println("Email Notification:");
        notifier.send();
    }
}

```

```

        System.out.println("Email + SMS Notification:");
        Notifier smsNotifier =new SMSNotifierDecorator(new EmailNotifier());
        smsNotifier.send();

        System.out.println("Email + SMS + Slack Notification:");
        Notifier allNotifier =new SlackNotifierDecorator(new SMSNotifierDecorator(new
EmailNotifier()));
        allNotifier.send();
    }
}

//      Create an interface Notifier with a method send().
interface Notifier{
    void send();
}

//      Create a class EmailNotifier that implements Notifier.
class EmailNotifier implements Notifier {
    @Override
    public void send() {
        System.out.println("Sending notification via Email");
    }
}

//      Create abstract decorator class NotifierDecorator that implements Notifier
abstract class NotifierDecorator implements Notifier {

    //holds a reference to a Notifier object.
    protected Notifier notifier;
    public NotifierDecorator(Notifier notifier) {
        this.notifier = notifier;
    }
}

//      Create concrete decorator classes like SMSNotifierDecorator,
SlackNotifierDecorator that extend NotifierDecorator.
class SMSNotifierDecorator extends NotifierDecorator{
    public SMSNotifierDecorator(Notifier notifier) {
        super(notifier);
    }

    @Override
    public void send() {
        notifier.send();
        System.out.println("Sending notification via SMS");
    }
}

```

```

class SlackNotifierDecorator extends NotifierDecorator{
    public SlackNotifierDecorator(Notifier notifier){
        super(notifier);
    }
    @Override
    public void send(){
        notifier.send();
        System.out.println("Sending notification via Slack");
    }
}

```

Output

```

Email Notification:
Sending notification via Email

Email + SMS Notification:
Sending notification via Email
Sending notification via SMS

Email + SMS + Slack Notification:
Sending notification via Email
Sending notification via SMS
Sending notification via Slack

```

Exercise 6: Implementing the Proxy Pattern

```

// Testing the Proxy Implementation

public class Main{
    public static void main(String[] args){
        Image image = new ProxyImage("nature.jpg");
        System.out.println("Image object created.");
        System.out.println("\nFirst Display:");
        image.display();
        System.out.println("\nSecond Display:");
        image.display();
    }
}

```


// Create an interface Image with a method display().

```
interface Image{  
    void display();  
}
```

// Create a class ReallImage that implements Image and loads an image from a remote server.

```
class ReallImage implements Image{  
    private String fileName;  
    public ReallImage(String fileName) {  
        this.fileName = fileName;  
        loadFromServer();  
    }  
  
    private void loadFromServer() {  
        System.out.println("Loading " + fileName+ " image from remote server");  
    }  
  
    @Override  
    public void display() {  
        System.out.println("Displaying image: " + fileName);  
    }  
}
```

// Create a class ProxyImage that implements Image and holds a reference to ReallImage.

```
class ProxyImage implements Image{  
    private ReallImage reallImage;  
    private String fileName;  
    public ProxyImage(String fileName) {  
        this.fileName = fileName;  
    }  
}
```

// Implement lazy initialization and caching in ProxyImage.

```
@Override  
public void display() {  
    if (reallImage == null) {  
        reallImage = new ReallImage(fileName);  
    }  
    reallImage.display();  
}  
}
```

Output

Image object created.

First Display:

Loading nature.jpg image from remote server

Displaying image: nature.jpg

Second Display:

Displaying image: nature.jpg

Exercise 7: Implementing the Observer Pattern

```
import java.util.*;

// Testing the Observer Implementation
public class Main {
    public static void main(String[] args) {
        StockMarket stockMarket = new StockMarket();

        Observer mobile = new MobileApp();
        Observer web = new WebApp();
        stockMarket.register(mobile);
        stockMarket.register(web);
        System.out.println("Stock Price Updated:\n");

        stockMarket.setStockPrice("Cognizant", 4200.50);
        stockMarket.setStockPrice("INFOSYS", 1850.75);
    }
}

// Create an interface Stock with methods to register, deregister, and notify observers.
interface Stock {
    void register(Observer observer);
    void deregister(Observer observer);
    void notifyObservers();
}

// Create an interface Observer with a method update().
interface Observer{
    void update(String name, double price);
}
```

```
}
```

// Create a class StockMarket that implements Stock and maintains a list of observers.

```
class StockMarket implements Stock{
    private List<Observer> observers = new ArrayList<>();
    private String name;
    private double price;
```

```
    public void setStockPrice(String name, double price) {
        this.name = name;
        this.price = price;
        notifyObservers();
    }
```

```
    @Override
    public void register(Observer observer) {
        observers.add(observer);
    }
```

```
    @Override
    public void deregister(Observer observer) {
        observers.remove(observer);
    }
```

```
    @Override
    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(name, price);
        }
    }
}
```

// Create classes MobileApp, WebApp that implement Observer.

```
class MobileApp implements Observer{
    @Override
    public void update(String name, double price) {
        System.out.println("Mobile App Notification: " + name + " : Rs." + price);
    }
}
```

```
class WebApp implements Observer{
    @Override
    public void update(String name, double price){
        System.out.println("WebApp Notification: " + name + " :Rs." + price);
    }
}
```

Output

Stock Price Updated:

Mobile App Notification: Cognizant : Rs.4200.5

WebApp Notification: Cognizant :Rs.4200.5

Mobile App Notification: INFOSYS : Rs.1850.75

WebApp Notification: INFOSYS :Rs.1850.75

Exercise 8: Implementing the Strategy Pattern

// testing the Strategy Implementation

```
public class Main{
```

```
    public static void main(String []args){
```

```
        PaymentContext paymentContext = new PaymentContext();
```

```
        paymentContext.setPaymentStrategy(new CreditCardPayment());
```

```
        paymentContext.executePayment(5000);
```

```
        paymentContext.setPaymentStrategy(new PayPalPayment());
```

```
        paymentContext.executePayment(10000);
```

```
    }
```

```
}
```

// Create an interface PaymentStrategy with a method pay().

```
interface PaymentStrategy{
```

```
    void pay(double amount);
```

```
}
```

// Create classes CreditCardPayment, PayPalPayment that implement PaymentStrategy.

```
class CreditCardPayment implements PaymentStrategy{
```

```
    @Override
```

```
public void pay(double amount){  
    System.out.println("paying amount of "+ amount + " through Credit Card");  
}  
}
```

```
class PayPalPayment implements PaymentStrategy{  
    @Override  
    public void pay(double amount){  
        System.out.println("paying amount of "+ amount + "through PayPal");  
    }  
}
```

// Create a class PaymentContext that holds a reference to PaymentStrategy and a method to execute the strategy.

```
class PaymentContext{  
    private PaymentStrategy strategy;  
    public void setPaymentStrategy(PaymentStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void executePayment(double amount) {  
        strategy.pay(amount);  
    }  
}
```

Output

```
paying amount of 5000.0 through Credit Card  
paying amount of 10000.0through PayPal
```

Exercise 9: Implementing the Command Pattern

//testing the Command Implementation

```
public class Main{  
    public static void main(String[] args){  
        // I have created Receiver  
        Light light = new Light();  
        Command lightOn = new LightOnCommand(light);  
        Command lightOff = new LightOffCommand(light);  
  
        //I have created Invoker  
        RemoteControl remoteControl = new RemoteControl();  
        remoteControl.setCommand(lightOn);  
        remoteControl.pressButton();  
  
        remoteControl.setCommand(lightOff);  
        remoteControl.pressButton();  
    }  
}
```

// Create an interface Command with a method execute().

```
interface Command{  
    void execute();  
}
```

// Create classes LightOnCommand, LightOffCommand that implement Command.

```
class LightOnCommand implements Command{  
    @Override  
    public void execute(){  
        light.turnOn();  
    }  
}
```

```
private Light light;

public LightOnCommand(Light light) {
    this.light = light;
}
}
```

```
class LightOffCommand implements Command{
    @Override
    public void execute(){
        light.turnOff();
    }
    private Light light;
    public LightOffCommand(Light light){
        this.light = light;
    }
}
```

// Create a class RemoteControl that holds a reference to a Command and a method to execute the command.

```
class RemoteControl{
    private Command command;

    public void setCommand(Command command) {
        this.command = command;
    }

    public void pressButton() {
        command.execute();
    }
}
```

// Create a class Light with methods to turn on and off.

```
class Light{  
    public void turnOn(){  
        System.out.println("Light is turned On");  
    }  
    public void turnOff(){  
        System.out.println("Light is turned Off");  
    }  
}
```

Output

```
Light is turned On  
Light is turned Off
```

Exercise 10: Implementing the MVC Pattern

// Create a class Student with attributes like name, id, and grade.

```
class Student {  
    private String name;  
    private int id;  
    private String grade;  
  
    public Student(String name, int id, String grade) {  
        this.name = name;  
        this.id = id;  
        this.grade = grade;  
    }  
    public String getName() {  
        return name;  
    }  
}
```



```

public int getId() {
    return id;
}
public String getGrade() {
    return grade;
}
public void setName(String name) {
    this.name = name;
}
public void setGrade(String grade) {
    this.grade = grade;
}
}

```

// Create a class StudentView with a method displayStudentDetails()

```

class StudentView{
    public void displayStudentDetails(String name,int id,String grade){
        System.out.println("Student Details");
        System.out.println("-----");
        System.out.println("Name : " + name);
        System.out.println("ID   : " + id);
        System.out.println("Grade : " + grade);
    }
}

```

// Create a class StudentController that handles the communication between the model and the view

```

class StudentController {
    private Student model;
    private StudentView view;
    public StudentController(Student model,StudentView view) {

```

```

        this.model = model;

        this.view = view;
    }

    public void setStudentName(String name) {
        model.setName(name);
    }

    public void setStudentGrade(String grade) {
        model.setGrade(grade);
    }

    public void updateView() {
        view.displayStudentDetails(model.getName(),model.getId(),model.getGrade());
    }
}

```

//testing MVC

```

public class Main {

    public static void main(String[] args) {

        Student model = new Student("Tarun",101,"A");

        StudentView studentView =new StudentView();

        StudentController studentController =new StudentController(model,studentView);

        System.out.println("Initial Student Record\n");

        studentController.updateView();


        System.out.println();

        studentController.setStudentName("Sai");

        studentController.setStudentGrade("A+");

        System.out.println("Updated Student Record\n");

        studentController.updateView();

    }

}

```

Output

Initial Student Record

Student Details

Name : Tarun

ID : 101

Grade : A

Updated Student Record

Student Details

Name : Sai

ID : 101

Grade : A+

Exercise 11: Implementing Dependency Injection

//Testing the Dependency Injection Implementation

```
class Main{
```

```
    public static void main(String[] args){
```

```
        CustomerRepository repo = new CustomerRepositoryImpl();
```

```
        CustomerService service =new CustomerService(repo);
```

```
        service.displayCustomer(101);
```

```
        System.out.println();
```

```
        service.displayCustomer(102);
```

```
    }
```

```
}
```

// Create an interface CustomerRepository with methods like findCustomerById()

```
interface CustomerRepository{  
    String findCustomerById(int id);  
}
```

// Create a class CustomerRepositoryImpl that implements CustomerRepository.

```
class CustomerRepositoryImpl implements CustomerRepository {  
    @Override  
    public String findCustomerById(int id){  
        if(id == 101){  
            return "Tarun";  
        }  
        if(id==102){  
            return "Sai";  
        }  
        return "Customer not found";  
    }  
}
```

// Create a class CustomerService that depends on CustomerRepository

```
class CustomerService{  
    private CustomerRepository repo;  
    public void displayCustomer(int id) {  
        String customer = repo.findCustomerById(id);  
        System.out.println("Customer ID : " + id);  
        System.out.println("Customer Name : " + customer);  
    }  
    // Use constructor injection to inject CustomerRepository into CustomerService  
    public CustomerService(CustomerRepository repo){  
        this.repo = repo;  
    }  
}
```

Output

Customer ID : 101

Customer Name : Tarun

Customer ID : 102

Customer Name : Sai